TRP 1 Week 4: The Brownfield Cartographer

Engineering Codebase Intelligence Systems for Rapid FDE Onboarding in Production Environments

The Business Objective

The Day-One Problem That Defines FDE Success

In Week 1, you built governance over code you were generating. In Week 2, you built a system to judge code you could read. Now you face the hardest real-world FDE scenario: You have been embedded at a client. You have 72 hours to become useful. The codebase has 800,000 lines of Python, Java, and Spark SQL. The original engineers are unavailable. The documentation is three years out of date. The data pipelines are running, but nobody is entirely sure how.

This is not a hypothetical. This is the default condition of every brownfield FDE engagement. The ability to rapidly develop a working mental model of an unfamiliar production system is the single highest-leverage skill in forward deployment. It cannot be fully automated—but it can be dramatically accelerated by the right tools.

The Cognitive Bottleneck of Brownfield Work

✗ **Navigation Blindness:** You can grep for function names but you cannot see the system. You do not know which files matter, which are dead code, which are the critical path, or how data flows from ingestion to output.

✗ **Contextual Amnesia:** Every conversation with the codebase starts from zero. The LLM you use has no persistent model of the project's architecture. You spend 40% of your time re-explaining context that should already be known.

✗ **Dependency Opacity:** In a data engineering codebase, the most important questions are: What produces this dataset? What consumes it? What breaks if this table changes? Standard tools cannot answer these questions for mixed SQL/Python/YAML pipelines.

✗ **Silent Debt:** The difference between what the code does and what the documentation says it does grows with every commit. You inherit not just the code but every lie told by stale docs.

The Master Thinker Philosophy

The FDE does not memorize codebases. The FDE builds instruments that make codebases legible. A cartographer does not need to walk every road to produce a map—they build systematic methods for extracting structure and representing it. Your task is to build such an instrument: a Codebase Intelligence System that produces a living, queryable map of any production codebase.

This challenge is deliberately scoped to data science and data engineering codebases—the dominant environment of FDE work—because they have unique structural properties: pipelines, DAGs, schemas, data lineage, mixed polyglot stacks (Python + SQL + YAML + notebooks). Your system must understand these structures, not just index them as text.2. Your Mission
You will build The Brownfield Cartographer—a multi-agent codebase intelligence system that ingests any
GitHub repository (or local path) and produces a living, queryable knowledge graph of the system's architecture, data flows, and semantic structure.

The Cartographer's Outputs

The System Map: A visual and queryable architectural overview: modules, services, entry points, critical path identification, and dead code detection.

The Data Lineage Graph: For data engineering codebases: the full DAG of data flow from source tables to output datasets, crossing Python, SQL, and config boundaries.

The Semantic Index: A vector-indexed, LLM-searchable knowledge base where every function, class, and module has a purpose-description grounded in its actual code—not its stale docstring.

The Onboarding Brief: An auto-generated 'FDE Day-One Brief': a structured document that answers the five questions every new FDE needs answered immediately.

The Living Context (CODEBASE.md): A persistent, auto-updating context file that can be injected into any AI coding agent to give it instant architectural awareness. This is the evolution of Week 1's CLAUDE.md.3. Mandatory Research & Conceptual Foundation

The FDE working in data engineering and data science environments must be fluent in these technical domains. Do not approach this as research for a homework assignment. Approach it as the briefing you would give yourself before a client engagement.

Static Analysis & Code Intelligence

tree-sitter https://tree-sitter.github.io/tree-sitter/
The production-grade parser generator used by GitHub, Neovim, and VS Code. Supports 50+ languages including Python, SQL, YAML, and JavaScript. Critical concept: query the AST using tree-sitter's S-expression query syntax. This is not grep—it is structural code understanding. You will use it to extract function signatures, class hierarchies, import graphs, and SQL table references from mixed-language codebases.

jedi / rope https://jedi.readthedocs.io
Python static analysis libraries that provide semantic understanding beyond AST: type inference, name resolution across files, definition lookup. Relevant when tree-sitter gives you syntax and you need semantics.

sqlglot https://github.com/tobymao/sqlglot
A production-grade SQL parser and transpiler supporting 20+ SQL dialects. This is how you parse the SELECT, FROM, JOIN, CTE dependencies in dbt models, Spark SQL jobs, and raw .sql files to build data lineage. Study: how to extract table dependencies from a SQL AST.

NetworkX / graph-tool https://networkx.org
Graph construction and analysis for building dependency graphs and data lineage DAGs. Key algorithms you will use: topological sort (pipeline order), strongly connected components (circular dependencies), PageRank (identifying critical/high-impact nodes).

Data Engineering & AI Engineering Patterns

Data Lineage

The discipline of tracking how data flows, transforms, and moves through a system. Study: OpenLineage specification (https://openlineage.io), dbt's lineage graph model, and Apache Atlas. Key insight: lineage is not just about tables—it includes transformation logic, schema evolution, and column-level provenance. An FDE who cannot reconstruct lineage from existing code cannot answer the question 'Why does this metric look wrong today?'

LLMs as Code Intelligence Tools

Study: Microsoft's CodeBERT, GitHub Copilot's architecture, and the emerging pattern of repo-level context injection.
Key paper: RepoFusion (how to build repo-aware code models).
Practical insight: the challenge is not summarizing individual functions but maintaining a coherent model of how they relate. This is a context engineering problem.

dbt (data build tool)

The dominant framework for data transformation in modern data stacks. Understanding dbt's DAG structure, ref() system, and schema.yml metadata format is essential for any data engineering FDE engagement. Your Cartographer must be able to parse a dbt project as a first-class input.

The Five FDE Day-One Questions

Based on patterns from forward-deployed engineering engagements, these are the questions that must be answered in the first 72 hours: (1) What is the primary data ingestion path? (2) What are the 3-5 most critical output datasets/endpoints? (3) What is the blast radius if the most critical module fails? (4) Where is the business logic concentrated vs. distributed? (5) What has changed most frequently in the last 90 days (git velocity map)?

The Architecture: The Intelligence System

The Cartographer is a multi-agent system with four specialized analysis agents, a knowledge graph as its central data store, and a query interface that allows both natural language and structured interrogation of the codebase.

Agent 1: The Surveyor (Static Structure Analyst)

Performs deep static analysis of the codebase using tree-sitter for language-agnostic AST parsing. Builds the structural skeleton of the system. What it extracts per file:

- Module graph: which files import which (cross-language: Python imports + relative path resolution)
- Public API surface: all exported/public functions and classes with their signatures
- Complexity signals: cyclomatic complexity, lines of code, comment ratio
- Change velocity: git log --follow analysis to identify which files change most frequently
- Dead code candidates: exported symbols with no internal or external import references

Agent 2: The Hydrologist (Data Flow & Lineage Analyst)

Specialized for data engineering codebases. Constructs the data lineage DAG by analyzing data sources, transformations, and sinks across all languages in the repo. Supported input patterns:

- Python: pandas read/write operations, SQLAlchemy queries, PySpark transformations
- SQL / dbt: sqlglot-parsed table dependencies from SELECT/FROM/JOIN/CTE chains
- YAML/Config: Airflow DAG definitions, dbt schema.yml, Prefect flow definitions
- Notebooks: Jupyter .ipynb files parsed for data source references and output paths

Output: A DataLineageGraph (NetworkX DiGraph) where nodes are datasets/tables and edges are transformations with transformation_type, source_file, and line_range metadata. This graph must answer: 'Show me all upstream dependencies of table X' and 'What would break if I change the schema of table Y?'

Agent 3: The Semanticist (LLM-Powered Purpose Analyst)

Uses LLMs to generate semantic understanding of code that static analysis cannot provide. This is not summarization—it is purpose extraction grounded in implementation evidence.
Core tasks:

- For each module: generate a Purpose Statement (what this module does, not how) based on its code, not its docstring.
- Flag if the docstring contradicts the implementation.
- Identify Business Domain boundaries: cluster modules into inferred domains (e.g., 'ingestion','transformation', 'serving', 'monitoring') based on semantic similarity.
- Generate the Five FDE Day-One Answers by synthesizing Surveyor + Hydrologist output with LLM reasoning over the full architectural context.
- Cost discipline: use a fast, cheap model (Gemini Flash / Mistral via OpenRouter) for bulk semantic extraction. Reserve expensive models for synthesis tasks only.

Agent 4: The Archivist (Living Context Maintainer)

Produces and maintains the system's outputs as living artifacts that can be re-used and updated as the codebase evolves. This agent is the direct evolution of Week 1's CLAUDE.md and Week 2's Audit Report pattern. Artifacts produced:

- **CODEBASE.md:** The living context file. Structured for direct injection into AI coding agents. Sections: Architecture Overview, Critical Path, Data Sources & Sinks, Known Debt, Recent Change Velocity, and Module Purpose Index.
- **onboarding_brief.md:** The Day-One Brief answering the five FDE questions with evidence citations.

- **lineage_graph.json:** The serialized DataLineageGraph for downstream tooling.
- **semantic_index/:** Vector store of all module Purpose Statements for semantic search.
- **cartography_trace.jsonl:** Audit log of every analysis action (mirrors Week 1's agent_trace.jsonl).

The Query Interface: The Navigator Agent

A LangGraph agent with four tools that allows both exploratory investigation and precise structured querying of the codebase knowledge graph:

Tool	Query Type	Example
find_implementation(concept)	Semantic	"Where is the revenue calculation logic?"
trace_lineage(dataset, direction)	Graph	"What produces the daily_active_users table?"
blast_radius(module_path)	Graph	"What breaks if I change src/transforms/revenue.py?"
explain_module(path)	Generative	"Explain what src/ingestion/kafka_consumer.py does"

The Knowledge Graph Schema

The central data store is a knowledge graph stored as a combination of a NetworkX graph (for structure and lineage) and a vector store (for semantic search). All nodes and edges must conform to these Pydantic schemas:

Node Types

ModuleNode: path, language, purpose_statement, domain_cluster, complexity_score, change_velocity_30d, is_dead_code_candidate, last_modified

DatasetNode: name, storage_type [table|file|stream|api], schema_snapshot, freshness_sla, owner, is_source_of_truth

FunctionNode: qualified_name, parent_module, signature, purpose_statement, call_count_within_repo, is_public_api
TransformationNode: source_datasets, target_datasets, transformation_type, source_file, line_range, sql_query_if_applicable

Edge Types

IMPORTS: source_module → target_module. Weight = import_count.
PRODUCES: transformation → dataset. Captures data lineage.
CONSUMES: transformation → dataset. Captures upstream dependencies.
CALLS: function → function. For call graph analysis.
CONFIGURES: config_file → module/pipeline. YAML/ENV relationship.

Implementation Curriculum

The following phases provide direction. A complete, working system requires engineering decisions and gap-filling beyond what is described here. Innovation in handling real-world codebase messiness is expected and rewarded.

Phase 0: The Target Codebase Selection & Reconnaissance

Goal: Choose a real brownfield target and build a preliminary mental model before automation.

1. Select a real open-source data engineering codebase as your primary target. Recommended candidates: Apache Airflow (Python + YAML), dbt's jaffle_shop (dbt + SQL + Python), Meltano (Python + YAML), or any company's public data platform repository. The codebase must have: 50+ files, multiple languages, SQL and Python, and be a real production system (not a tutorial repo).
2. Spend 30 minutes manually exploring the repo. Answer the Five FDE Day-One Questions by hand. Write your answers in RECONNAISSANCE.md. This becomes the ground truth you measure your system's output against.
3. Document: what was hardest to figure out manually? Where did you get lost? This informs your architecture's priorities.
4. Deliverable: RECONNAISSANCE.md with manual Day-One answers + difficulty analysis.

Phase 1: The Surveyor Agent (Static Structure)

Goal: Build the structural analysis layer using tree-sitter.

1. Install tree-sitter and the grammars for Python, SQL, YAML, and JavaScript/TypeScript. Write a LanguageRouter that selects the correct grammar based on file extension.
2. Implement analyze_module(path) that returns a ModuleNode: extract imports (Python import statements + relative paths), public functions (decorated with leading underscores stripped), and class definitions with inheritance.
3. Implement extract_git_velocity(path, days=30): parse git log output to compute change frequency per file. Identify the 20% of files responsible for 80% of changes (the high-velocity core).
4. Build the module import graph as a NetworkX DiGraph. Run PageRank to identify the most 'imported' modules (architectural hubs). Identify strongly connected components (circular dependencies).
5. Write the graph to .cartography/module_graph.json using NetworkX's JSON serializer.

Phase 2: The Hydrologist Agent (Data Lineage)

Goal: Build the data lineage layer for mixed Python/SQL/YAML codebases.

1. Implement PythonDataFlowAnalyzer: use tree-sitter to find pandas read_csv/read_sql, SQLAlchemy execute(), PySpark read/write calls. Extract the dataset names/paths as strings. Handle f-strings and variable references gracefully (log as 'dynamic reference, cannot resolve').
2. Implement SQLLineageAnalyzer using sqlglot: parse .sql files and dbt model files. Extract the full table dependency graph from SELECT/FROM/JOIN/WITH (CTE) chains. Support at minimum: PostgreSQL, BigQuery, Snowflake, and DuckDB dialects.
3. Implement DAGConfigAnalyzer: parse Airflow DAG files or dbt schema.yml to extract pipeline topology from configuration (not just code).
4. Merge all three analyzers into the DataLineageGraph. Implement blast_radius(node): BFS/DFS from a node to find all downstream dependents.

5. Implement find_sources() and find_sinks(): nodes with in-degree=0 and out-degree=0 in the lineage graph. These are the entry and exit points of the data system.

Phase 3: The Semanticist Agent (LLM-Powered Analysis)

Goal: Add semantic understanding that static analysis cannot provide.

1. Build a ContextWindowBudget: before calling any LLM, estimate token count and track cumulative spend. Implement a tiered model selection: use gemini-flash for bulk module summaries, reserve claude or gpt-4 for synthesis.
2. Implement generate_purpose_statement(module_node): prompt the LLM with the module's code (not docstring) and ask for a 2-3 sentence purpose statement that explains business function, not implementation detail. Cross-reference with the existing docstring—flag discrepancies as 'Documentation Drift'.
3. Implement cluster_into_domains(): embed all Purpose Statements, run k-means clustering (k=5-8), and label each cluster with an inferred domain name. This produces the Domain Architecture Map.
4. Implement answer_day_one_questions(): a synthesis prompt that feeds the full Surveyor + Hydrologist output and asks the LLM to answer the Five FDE Questions with specific evidence citations (file paths and line numbers).

Phase 4: The Archivist, Living Context & Query Interface

Goal: Produce the final deliverables and the Navigator query agent.

1. Implement generate_CODEBASE_md(): structure the living context file to be immediately useful when injected into an AI coding agent. Sections must include: Architecture Overview (1 paragraph), Critical Path (top 5 modules by PageRank), Data Sources & Sinks (from Hydrologist), Known Debt (circular deps + doc drift flags), and High-Velocity Files (files changing most frequently = likely pain points).
2. Build the Navigator LangGraph agent with the four tools. Every answer must cite evidence: the source file, the line range, and the analysis method that produced it (static analysis vs. LLM inference—this distinction matters for trust).
3. Implement the cartography_trace.jsonl: log every agent action, evidence source, and confidence level. This is your Week 1 audit pattern applied to intelligence gathering.

4. Build an incremental update mode: if git log shows new commits since last run, re-analyze only the changed files rather than the full codebase. This makes the Cartographer practical for ongoing FDE Engagements.

Required Target Codebases

Your system must produce results for at least two of the following real-world codebases. These represent the types of systems you will encounter in actual FDE deployments:

Primary: dbt jaffle_shop or any dbt project

https://github.com/dbt-labs/jaffle_shop

The canonical dbt example project. Mixed SQL + YAML + Python. Your system must extract the full dbt DAG as a data lineage graph. Verification: your lineage graph must match dbt's own built-in lineage visualization.

Primary: Apache Airflow example DAGs

https://github.com/apache/airflow (examples/)

Real production DAG definitions. Python + YAML. Your system must identify the pipeline topology, dependencies, and data sources from Airflow operator definitions.

Stretch: A real company's open-source data platform

e.g., Airbnb's Minerva, Spotify's Backstage data plugins, Stripe's open-source tools

An actual brownfield system with undocumented complexity. The highest-value demonstration: your Day-One Brief on a real enterprise repo that no one on the team has read before.

Your Own Week 1 Submission

Local path to your Week 1 code Self-referential validation. Run the Cartographer on your own Week 1 codebase. Compare the generated CODEBASE.md against your own ARCHITECTURE_NOTES.md. Discrepancies are either bugs in your Cartographer or gaps in your Week 1 documentation.

Proof of Execution — The Demo Protocol

To pass, you must submit a video (max 6 minutes) following this sequence. The first 3 minutes are required. The last 3 minutes demonstrate mastery.

Step 1: The Cold Start (Required)
Point the Cartographer at a codebase no one on your team has discussed. Run the analysis. Show the CODEBASE.md being generated. Time it.

Step 2: The Lineage Query (Required)
Ask: 'What upstream sources feed this output dataset?' Show the DataLineageGraph traversal returning the answer with file:line citations.

Step 3: The Blast Radius (Required)
Pick a module. Run blast_radius(). Show the dependency graph of everything that would break if that module changed its interface.

Step 4: The Day-One Brief (Mastery)
Show your onboarding_brief.md. Read the Five FDE Day-One Answers aloud. Then pull up the codebase and verify at least two answers by navigating to the cited file and line. The answer must be correct.

Step 5: The Living Context Injection (Mastery)
Open a new AI coding agent session. Inject CODEBASE.md as the system prompt context. Ask the agent a question about the codebase architecture. Show that it answers correctly because of the injected context. Compare with the same question without context injection.

Step 6: The Self-Audit (Mastery)
Run the Cartographer on your own Week 1 repo. Show one discrepancy between your ARCHITECTURE_NOTES.md and the auto-generated CODEBASE.md. Explain what it means.

Deliverables

Interim -- Thursday March 12, 03:00 UTC

GitHub Code:

- src/cli.py -- entry point, takes repo path (local or GitHub URL), runs analysis
- src/orchestrator.py -- wires Surveyor + Hydrologist in sequence, serializes outputs to .cartography/
- src/models/ -- all Pydantic schemas (Node types, Edge types, Graph types)
- src/analyzers/tree_sitter_analyzer.py -- multi-language AST parsing with LanguageRouter
- src/analyzers/sql_lineage.py -- sqlglot-based SQL dependency extraction
- src/analyzers/dag_config_parser.py -- Airflow/dbt YAML config parsing
- src/agents/surveyor.py -- module graph, PageRank, git velocity, dead code candidates
- src/agents/hydrologist.py -- DataLineageGraph, blast_radius, find_sources/find_sinks
- src/graph/knowledge_graph.py -- NetworkX wrapper with serialization
- pyproject.toml with locked deps (uv)
- README.md -- how to install and run analysis (at least analyze command documented)
- Cartography Artifacts (at least 1 target codebase) containing the following**:**
    - .cartography/module_graph.json
    - .cartography/lineage_graph.json (partial is acceptable -- at minimum SQL lineage via sqlglot)

Single PDF Report containing:

1. RECONNAISSANCE.md content (manual Day-One analysis for chosen target)
2. Architecture diagram of the four-agent pipeline with data flow
3. Progress summary: what's working, what's in progress
4. Early accuracy observations: does the module graph look right? Does the lineage graph match reality?
5. Known gaps and plan for final submission

Final -- Sunday March 15, 03:00 UTC

GitHub Code (full system):

- src/cli.py -- updated with subcommands: analyze (full pipeline) and query (Navigator interactive mode)
- src/orchestrator.py -- updated to run full pipeline: Surveyor -> Hydrologist -> Semanticist -> Archivist
- src/models/ -- all Pydantic schemas (Node types, Edge types, Graph types)
- src/analyzers/tree_sitter_analyzer.py -- multi-language AST parsing with LanguageRouter
- src/analyzers/sql_lineage.py -- sqlglot-based SQL dependency extraction
- src/analyzers/dag_config_parser.py -- Airflow/dbt YAML config parsing
- src/agents/surveyor.py -- module graph, PageRank, git velocity, dead code candidates
- src/agents/hydrologist.py -- DataLineageGraph, blast_radius, find_sources/find_sinks
- src/agents/semanticist.py -- LLM purpose statements, doc drift detection, domain clustering, Day-One question answering, ContextWindowBudget
- src/agents/archivist.py -- CODEBASE.md generation, onboarding brief, trace logging
- src/agents/navigator.py -- LangGraph agent with 4 tools (find_implementation, trace_lineage, blast_radius, explain_module)
- src/graph/knowledge_graph.py -- NetworkX wrapper with serialization
- Incremental update mode (re-analyze only changed files via git diff)
- pyproject.toml with locked deps (uv)
- README.md -- how to run against any GitHub URL, including both analyze and query modes

Cartography Artifacts (2+ target codebases, each with):

- .cartography/CODEBASE.md
- .cartography/onboarding_brief.md
- .cartography/module_graph.json
- .cartography/lineage_graph.json
- .cartography/cartography_trace.jsonl

Single PDF Report containing:

1. RECONNAISSANCE.md: manual Day-One analysis vs. system-generated output comparison
2. Architecture diagram of the four-agent pipeline (finalized)
3. Accuracy analysis: which Day-One answers were correct, which were wrong, and why
4. Limitations: what the Cartographer fails to understand, what remains opaque
5. FDE Applicability: one paragraph on how you would use this tool in a real client engagement
6. Self-audit results (Cartographer run on your own Week 1 repo, discrepancies explained)

Video Demo (max 6 min):

Minutes 1-3 (Required):

- **Step 1 -- Cold Start:** Point Cartographer at an unfamiliar codebase, run analyze, show CODEBASE.md being generated, time it
- **Step 2 -- Lineage Query:** Run query, ask upstream sources for an output dataset, show graph traversal with file:line citations
- **Step 3 -- Blast Radius:** Pick a module, run blast_radius(), show downstream dependency graph

Minutes 4-6 (Mastery):

- **Step 4 -- Day-One Brief:** Show onboarding_brief.md, read answers, verify 2+ answers by navigating to cited file and line
- **Step 5 -- Living Context Injection:** Inject CODEBASE.md into a fresh AI coding agent session, ask an architecture question, compare with/without context
- **Step 6 -- Self-Audit:** Run Cartographer on your own Week 1 repo, show a discrepancy between your docs and the generated output, explain it

Evaluation Rubric

Metric	1 -- The Vibe Coder	3 -- Competent Engineer	5 -- Master Thinker
Static Analysis Depth	Regex-based file scanning. Only indexes file names. No AST parsing. "Module graph" is a flat list.	tree-sitter AST parsing works for Python. Module import graph built. Basic PageRank applied.	Multi-language AST parsing (Python + SQL + YAML). Circular dependency detection.
Data Lineage Accuracy	No lineage graph. Only mentions "it reads from somewhere." Cannot answer upstream dependency questions.	Python dataframe read/write calls detected. Basic lineage graph built for simple cases.	Full mixed-language lineage: Python + sqlglot-parsed SQL + YAML config.
Semantic Intelligence	Docstring regurgitation. No distinction between "what code does" vs. "what docstring says." No domain clustering.	LLM Purpose Statements generated. Some documentation drift flags. Domain clusters attempted.	Purpose Statements demonstrably based on code analysis (not docstring). Full documentation drift detection.
FDE Readiness (Onboarding Value)	Output is a dump of file names. Cannot answer any Day-One question. CODEBASE.md is generic text.	CODEBASE.md is structured and mostly accurate. Can answer 3/5 Day-One questions.	All 5 Day-One questions answered correctly with evidence.
Engineering Quality	Single script. No Pydantic schemas. Hardcoded repo paths. Crashes on real-world codebases with unusual file structures.	Modular structure. Pydantic models for nodes. Handles most real-world cases with basic error handling.	Production-grade: graceful degradation on unparseable files (log + skip).

The Final Insight: Every tool you build for yourself is a tool you can deploy for a client. The Brownfield Cartographer is not just a training exercise—it is a deployable artifact that you can run on Day 1 of any FDE engagement, on any codebase, in any domain. The engineer who arrives at a client site and deploys this in the first hour becomes the person who understands the system before anyone else does.